PRD: Subscription Management System (Brightloop)

Oct 31, 2026 · @Ameture

1. Overview

Brightloop Software needs one platform that runs a subscription business end to end: products, plans, quotes, subscriptions, invoices, payments and revenue reporting. It replaces spreadsheets, hand-typed invoices and quarterly manual reports with a single database-backed system that three kinds of users share.

Today	Target
Subscriptions tracked in a spreadsheet	Central database with a Draft → Quotation → Confirmed → Active → Closed lifecycle
Renewals remembered by whoever notices	Scheduler bills on due dates and closes plans that end
Invoices typed by hand each month	Invoices generated from subscriptions, with taxes and discounts applied
Payments matched by scrolling bank statements	Payments recorded against invoices; paid vs outstanding always visible
Revenue reports built each quarter	Live dashboard with filters and a summary view

Goals

- Cover every scene in the brief: signup, plan launch, quote to subscription, billing day, discounts, payments, finance review.
- Make billing safe: nothing billed twice, nothing skipped.
- Keep every list and dashboard page under 2 seconds with thousands of subscriptions.
- Give four teammates clean, independent ownership so all four ship and commit.

Success metrics

Metric	Target
Page response with 10,000 subscriptions and 30,000 invoices seeded	Under 2 s (p95) for every list and dashboard page

Metric	Target
Billing run repeated twice on the same day	0 duplicate invoices, 0 skipped subscriptions
Invoices linked to a subscription and its payments	100%
Role checks	Customers cannot read another customer's data; staff cannot create staff
Demo	Full month at Brightloop runs live from a seeded local database

2. Users and roles

Three roles exist, and every API call and screen is checked against the role. Customers can only sign up themselves; staff accounts are created by the admin alone.

Capability	Admin	Staff	Customer
Add or disable staff members	Yes	No	No
Products, variants, plans	Create, edit, close	View	No
Tax rules	Create, edit	View	No
Discount rules	Create, edit	View	No
Quotation templates	Create, edit	Create, use	No
Customers	Full	Create, edit	Own profile only
Subscriptions	Full	Create, edit, move through lifecycle	View own; pause, renew or close own if the plan allows
Invoices	Full	Confirm, cancel, send, print	View and print own
Payments	Full	Record	View own
Dashboard and reports	Full	View	No

Account rules

- One account per email address (case-insensitive, unique constraint in the database).
- Password: more than 8 characters, with an uppercase letter, a lowercase letter and a special character. Stored as a bcrypt hash.
- Forgot password: a single-use reset link, valid for 30 minutes, is emailed to the registered address. The response is identical whether or not the email exists.
- A seeded admin account exists on first run; it is the only way the first staff member is created.

3. Functional requirements

The system has 11 modules; P0 is required for the demo, P1 should be done, P2 is nice to have. With only 6 hours, build every P0 row as thin as possible: forgot-password writes the reset link to Mailpit or the console, the portal is read-only, the dashboard shows the four headline numbers, and a P1 row starts only after every P0 row works end to end.

ID	Module	Requirement	Priority
AUTH-1	Accounts	Customer signup with unique email and password rules enforced in UI and API	P0
AUTH-2	Accounts	Login, logout, session expiry; forgot-password email with single-use reset link	P0
AUTH-3	Accounts	Role-based access (admin, staff, customer) on every route and endpoint	P0
AUTH-4	Accounts	Admin-only creation, disabling and role view of staff members	P0
CAT-1	Catalog	Product with name, type, sales price, cost price, notes	P0
CAT-2	Catalog	Product variants by attribute (e.g. Brand) with an extra price, such as Odoo adds 560	P0
PLAN-1	Plans	Plan with name, price, billing period (daily, weekly, monthly, yearly), minimum quantity, available-from and available-to dates	P0
PLAN-2	Plans	Per-plan switches set by admin: auto-close at end date, closable early, pausable, renewable	P0
PLAN-3	Plans	Pause, resume, close and renew actions that respect those switches	P1

ID	Module	Requirement	Priority
TAX-1	Tax	Tax rules (name, percentage or fixed amount, applies to products) maintained by admin	P0
DISC-1	Discounts	Admin-only discount rules: fixed or percentage, minimum purchase or quantity, start and end dates, usage limit	P0
DISC-2	Discounts	A discount applies to chosen products or to a whole subscription	P0
DISC-3	Discounts	Eligible discounts appear on the next invoice automatically and stop on expiry or when the usage limit is reached	P0
QT-1	Quotation templates	Template with plan, product lines and validity in days; staff start a quotation from it	P0
SUB-1	Subscriptions	Unique subscription number, customer, plan, start date, expiry date, payment terms, lines (product, quantity, unit price, taxes, amount)	P0
SUB-2	Subscriptions	Lifecycle Draft → Quotation → Confirmed → Active → Closed, visible to the whole team with a status history	P0
SUB-3	Subscriptions	Search and filter by status, customer, plan, date range	P0
INV-1	Invoicing	Billing run creates invoices from due subscriptions with customer details, lines, taxes and totals prefilled	P0
INV-2	Invoicing	No invoice is created twice for the same subscription period, and none is skipped	P0
INV-3	Invoicing	Invoice lifecycle Draft → Confirmed → Paid, plus Cancel; send by email and print or PDF	P0
INV-4	Invoicing	Manual "Run billing now" for admin and staff, with a result summary	P1
PAY-1	Payments	Record payment with method, amount, date, note against an invoice; partial payments allowed	P0
PAY-2	Payments	Paid and outstanding views; overdue invoices highlighted with days overdue	P0

ID	Module	Requirement	Priority
REP-1	Reporting	Dashboard: active subscriptions, revenue collected, amount outstanding, overdue invoices	P0
REP-2	Reporting	Filters (date range, plan, customer, status) and a summary view; CSV export	P1
REP-3	Reporting	Monthly recurring revenue (MRR) trend	P1
PORT-1	Customer portal	Customers see only their own subscriptions, invoices and payments	P0
PORT-2	Customer portal	Customer actions on own subscription (pause, renew, close) when the plan allows	P1
SYS-1	Platform	Seed script that generates realistic data, including 10,000 subscriptions for performance tests	P0
SYS-2	Platform	Dashboard auto-refresh via polling or server-sent events	P2
SYS-3	Platform	Audit log of status changes and billing runs	P2

4. Lifecycles and business rules

Billing correctness depends on a few strict rules that every member must follow.

Subscription lifecycle

1. Draft: being prepared, editable.
2. Quotation: offer sent to the customer; valid for the template's number of days, then shown as expired and cannot be confirmed.
3. Confirmed: customer accepted; waiting for the start date.
4. Active: billing runs apply. Reaches Active on the start date or immediately if the start date has passed.
5. Closed: manually closed, or closed automatically when the plan ends and has auto-close switched on.

Only forward moves are allowed, except Active ↔ Paused when the plan is pausable. Each move is recorded with who and when.

Invoice lifecycle

1. Draft: created by the billing run, editable by staff.

2. Confirmed: locked and sent to the customer; due date = issue date + payment terms.
3. Paid: payments received cover the full total.

A Draft or Confirmed invoice can be cancelled; a cancelled invoice releases any discount usage it consumed. An invoice is overdue when it is Confirmed, past its due date and has a balance above zero.

Billing run (the "nothing twice, nothing skipped" rule)

- Runs daily from a scheduler and on demand. It selects Active subscriptions where the next invoice date is on or before today.
- A unique database constraint on (subscription, period start) makes a duplicate impossible, even if two runs overlap.
- Each subscription is billed in its own transaction: invoice, lines, discount usage and the new next invoice date commit together or not at all.
- If runs were missed, it loops until the next invoice date is in the future, so no period is skipped.
- Next invoice date adds one day, seven days, one month or one year; month-end dates clamp (31 Jan + 1 month = 28 or 29 Feb).

Price, discount and tax order

- Unit price = product sales price + variant extra price.
- Line amount = quantity × unit price, minus any eligible discount, then tax is added on the discounted amount.
- Rounding to 2 decimals happens per line; invoice totals are sums of rounded lines.
- A discount is eligible when today is inside its dates, usage is below the limit, and the minimum purchase or quantity is met. When several apply, the one giving the largest reduction wins (no stacking in v1).
- Usage count increases when the invoice is created and decreases if it is cancelled.

Payments

- Several payments can settle one invoice; the invoice becomes Paid when the sum is at least the total.
- A payment cannot exceed the outstanding balance and cannot be recorded against a cancelled or Draft invoice.

Monthly recurring revenue

Each active subscription is normalised to a month: monthly × 1, yearly ÷ 12, weekly × 52 ÷ 12, daily × 365 ÷ 12.

5. Data model

A relational database (PostgreSQL) with 17 tables; money is stored as exact decimals, never floats, and every foreign key is indexed.

Entity	Key fields	Relations
User	id, email (unique), password_hash, name, role (admin, staff, customer), is_active	One Customer profile if role is customer
PasswordResetToken	token_hash, expires_at, used_at	Belongs to User
Customer	company_name, contact_name, phone, billing_address, tax_id	One User; many Subscriptions and Invoices
Product	name, type, sales_price, cost_price, notes, is_active	Many Variants; many Taxes
ProductVariant	attribute (e.g. Brand), value (e.g. Odoo), extra_price	Belongs to Product
Plan	name, price, billing_period, min_quantity, available_from, available_to, auto_close, closable, pausable, renewable, status	Many Subscriptions and Templates
Tax	name, kind (percentage or fixed), rate, is_active	Many Products via ProductTax
Discount	name, kind (fixed or percentage), value, min_purchase, min_quantity, starts_on, ends_on, usage_limit, usage_count, scope (product or subscription)	Many Products via DiscountProduct
DiscountRedemption	discount_id, invoice_id, amount	Links a Discount to the Invoice that used it
QuotationTemplate	name, plan_id, validity_days	Many TemplateLines
QuotationTemplateLine	product_id, variant_id, quantity	Belongs to Template

Entity	Key fields	Relations
Subscription	number (unique), customer_id, plan_id, status, start_date, expiry_date, next_invoice_date, payment_term_days, quote_valid_until	Many SubscriptionLines, Invoices, status history rows
SubscriptionLine	product_id, variant_id, quantity, unit_price, tax_id, discount_id, amount	Belongs to Subscription
Invoice	number (unique), subscription_id, customer_id, period_start, period_end, status, issue_date, due_date, subtotal, discount_total, tax_total, total, amount_paid	Unique (subscription_id, period_start); many InvoiceLines and Payments
InvoiceLine	description, product_id, quantity, unit_price, discount_amount, tax_amount, amount	Belongs to Invoice; values copied so old invoices never change
Payment	invoice_id, method, amount, paid_on, note, recorded_by	Belongs to Invoice
AuditLog	actor_id, action, entity, entity_id, created_at	Written on status changes and billing runs

Indexes needed for the 2-second target

- Subscription: (status), (customer_id), (plan_id), (next_invoice_date).
- Invoice: (status, due_date), (customer_id), (subscription_id, period_start) unique.
- Payment: (invoice_id), (paid_on).

Migrations and seeds: all schema changes go through versioned migration files in Git; the seed script creates the admin, sample staff, customers, products, plans, taxes, discounts and the bulk data used for load tests.

6. API design

A versioned REST API under `/api/v1`, documented with OpenAPI at `/docs`, is the only way the web app reads or writes data.

Module	Endpoints	Access
Auth	`POST /auth/signup`, `/auth/login`, `/auth/logout`, `/auth/forgot-password`, `/auth/reset-password`; `GET /auth/me`	Public, then any signed-in user
Staff	`GET/POST /staff`, `PATCH /staff/:id` (disable, change details)	Admin
Customers	`GET/POST /customers`, `GET/PATCH /customers/:id`	Admin, staff
Products	`GET/POST /products`, `GET/PATCH/DELETE /products/:id`, `POST /products/:id/variants`, `PATCH/DELETE /variants/:id`	Read: admin, staff. Write: admin
Plans	`GET/POST /plans`, `PATCH /plans/:id`, `POST /plans/:id/pause`, `/resume`, `/close`, `/renew`	Read: admin, staff. Write: admin
Taxes	`GET/POST /taxes`, `PATCH/DELETE /taxes/:id`	Read: admin, staff. Write: admin
Discounts	`GET/POST /discounts`, `PATCH/DELETE /discounts/:id`	Read: admin, staff. Write: admin
Templates	`GET/POST /quotation-templates`, `PATCH/DELETE /quotation-templates/:id`	Admin, staff
Subscriptions	`GET/POST /subscriptions`, `POST /subscriptions/from-template/:templateId`, `GET/PATCH /subscriptions/:id`, `POST /subscriptions/:id/transition` with the target status	Admin, staff
Billing	`POST /billing/run`, `GET /billing/runs`	Admin, staff
Invoices	`GET /invoices`, `GET /invoices/:id`, `POST /invoices/:id/confirm`, `/cancel`, `/send`; `GET /invoices/:id/pdf`	Admin, staff
Payments	`POST /invoices/:id/payments`, `GET /payments`	Admin, staff

Module	Endpoints	Access
Reports	`GET /reports/summary`, `/reports/revenue?from&to&groupBy`, `/reports/overdue`, `/reports/mrr`	Admin, staff
Portal	`GET /portal/subscriptions`, `/portal/invoices`, `/portal/payments`, `POST /portal/subscriptions/:id/pause`, `/renew`, `/close`	Customer, own data only

Conventions

- List endpoints take `page`, `pageSize` (default 25), `sort`, `q` (search) and filter parameters; they return `{ data, page, pageSize, total }`.
- Errors use one shape: `{ error: { code, message, fields } }`, with `fields` mapping each invalid input to its message, so the UI can show it inline.
- Money is sent as strings with 2 decimals; dates as ISO 8601.
- State changes are explicit action endpoints, not free PATCHes of `status`, so every rule in section 4 has one place to live.

7. UI/UX and navigation

One consistent layout and colour system is used on every screen, and the menu shows only what the signed-in role may use.

Layout

- Staff and admin app: fixed left sidebar (collapses to a slide-out drawer below 768 px), top bar with page title, global search and user menu, content area with a 24 px gutter.
- Customer portal: same shell with a shorter menu, so the product feels like one system.
- Breakpoints: 360 px phone, 768 px tablet, 1280 px desktop. Tables become stacked cards on phones.

Design system (built once by the platform owner, used by everyone)

- One primary colour, one neutral scale and four status colours: draft (grey), pending (amber), active or paid (green), overdue or cancelled (red). Defined as design tokens, never hard-coded in screens.
- One font family, self-hosted so the app works offline. 8 px spacing scale.
- Shared components: data table with server-side pagination, sort and filters; status badge; form field with inline error; confirm dialog; toast; empty, loading and error states.

Menu by role

Group	Admin	Staff	Customer
Overview	Dashboard	Dashboard	Overview
Sales	Subscriptions, Quotation templates, Customers	Subscriptions, Quotation templates, Customers	My subscriptions
Billing	Invoices, Payments	Invoices, Payments	My invoices, My payments
Catalog	Products, Plans, Taxes, Discounts	Products, Plans, Taxes, Discounts (view only)	None
Reports	Revenue, Overdue, MRR	Revenue, Overdue, MRR	None
Settings	Staff, Profile	Profile	Profile

Key screens

Screen	Purpose
Sign up, login, forgot and reset password	Account entry with live password-rule checklist
Dashboard	Four headline cards (active subscriptions, revenue, outstanding, overdue), revenue chart, overdue list, filters
Subscription list and detail	Status filters, lifecycle stepper, lines table, invoices and history tabs
Invoice list and detail	Status, balance, confirm, cancel, send, print, record payment
Plan and product forms	Variant editor, plan switches (auto-close, closable, pausable, renewable)
Discount form	Type, value, scope, dates, usage counter
Billing run	Run now, see created, skipped and failed counts

8. Non-functional requirements

Performance, validation and offline operation are judged requirements, so each has a concrete test.

Performance (under 2 s per page)

- Server-side pagination on every list (25 rows default); no endpoint returns an unbounded list.
- Dashboard numbers come from SQL aggregates (COUNT, SUM, GROUP BY) on indexed columns, not from loading rows into the app.
- Verified (a P1 task, if time allows) by running the seed script with 10,000 subscriptions and 30,000 invoices, then load-testing the list and dashboard endpoints (autocannon or k6); the result goes in the README.

Input validation (client and server)

- One set of Zod schemas in the shared package validates forms in the browser and requests on the server, so the rules cannot drift apart. The server never trusts the browser.
- Inline field errors appear as the user leaves a field; submit is blocked until valid.

Field	Rule
Email	Valid format, trimmed, lowercased, unique
Password	More than 8 characters; at least one uppercase, one lowercase, one special character
Prices, costs, amounts	Decimal, 2 places, 0 or more; payment must be above 0 and not above the balance
Quantity	Whole number, at least the plan's minimum quantity
Dates	Valid; end on or after start; discount end on or after discount start
Percentage discount	Between 0 and 100
Names, notes	Required where marked; length limits; output escaped

Security

- Passwords hashed with bcrypt; sessions in httpOnly cookies or short-lived JWTs; login and reset endpoints rate-limited.
- Reset tokens stored hashed, single use, 30-minute expiry.
- Role check middleware on every route; customer queries always filtered by the signed-in customer's id.
- Parameterised queries only (through the ORM); secrets in `.env`, never in Git.

Offline and local-first

- Everything runs on a laptop with one command (Docker Compose, or local PostgreSQL if Docker is unavailable): database, API, web app, and a local mail catcher for reset and invoice emails.
- No runtime dependency on a CDN or cloud service: fonts, icons and libraries are bundled.
- PDFs and print views are generated locally.

Data

- All screens read from the API and database. Static JSON is allowed only for the first 30 minutes and is removed once the endpoint exists.

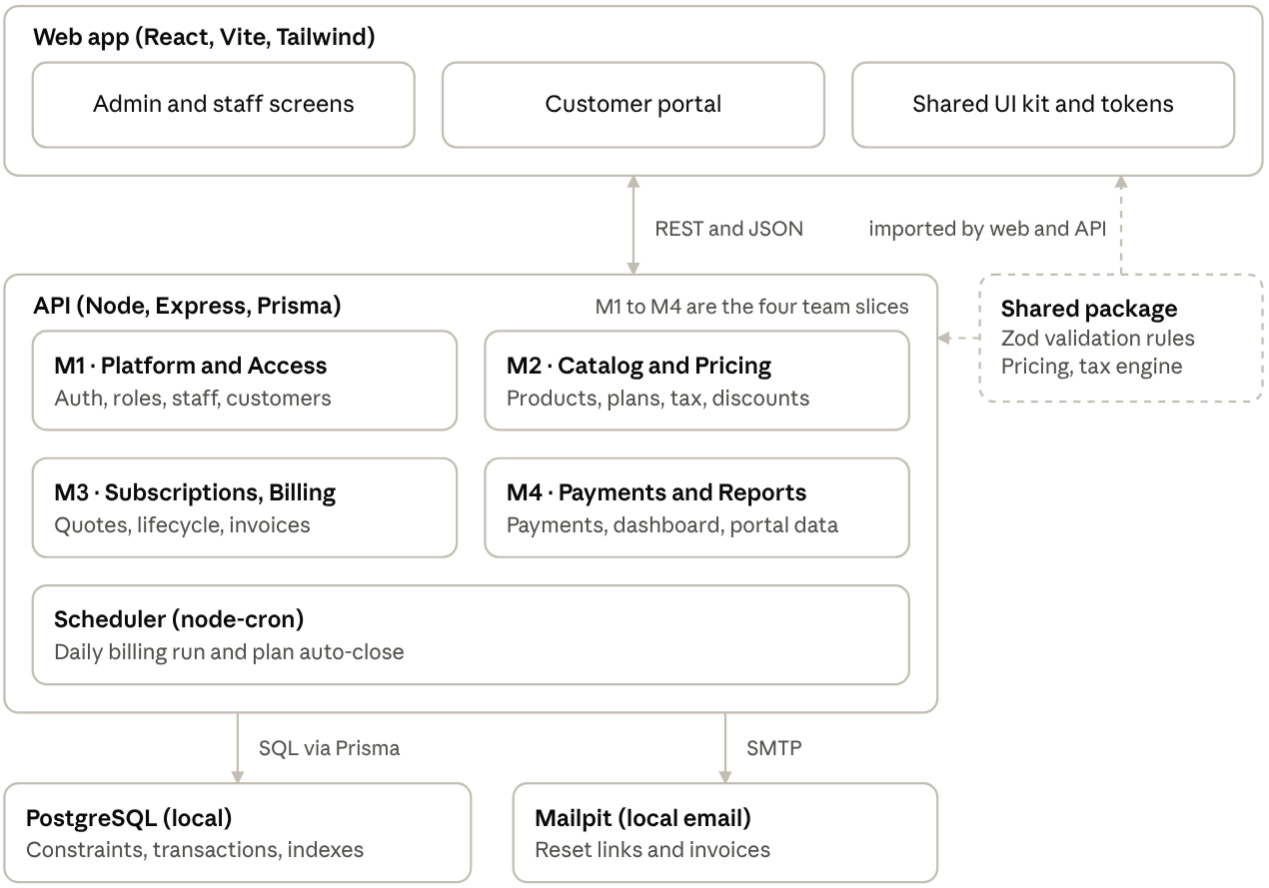
Quality

- Unit tests for the pricing, discount, tax and next-date logic; integration tests for the billing run (idempotency) and role access.

9. Tech stack and architecture

A single web app talking to one REST API and one local PostgreSQL database keeps the system simple enough for four people to finish. These are recommended defaults; swap a layer only if all four teammates already know the replacement.

Everything runs locally: one web app, one API, one database



architecture · web app, API, database

The web app, API and database all run on one laptop, and the shared package keeps validation and pricing identical in the browser and on the server.

Layer	Choice	Why
Web app	React with Vite, TypeScript, Tailwind CSS, React Router, TanStack Query	Fast to build, responsive utilities, cached server data
API	Node.js with Express, TypeScript	One language across the stack; shared validation code
Database	PostgreSQL (local, via Docker Compose)	Constraints, transactions and indexes that the billing rules need
ORM and migrations	Prisma	Versioned migrations and typed queries
Validation	Zod, in `packages/shared`	Same rules in browser and server
Auth	bcrypt, JWT in httpOnly cookie, role middleware	Simple and standard

Layer	Choice	Why
Scheduler	node-cron inside the API process	Daily billing and plan auto-close with no extra service
Email	Nodemailer with Mailpit locally	Reset links and invoices work offline
PDF and print	Print stylesheet first; a PDF library as a P1	No cloud dependency
Charts	Chart.js or Recharts, bundled	Dashboard visuals without a CDN
Tests	Vitest and Supertest	Pricing logic and billing idempotency
API docs	OpenAPI (Swagger UI)	Shared contract between teammates

What we deliberately skip: microservices, GraphQL and any AI feature, because they add setup time without helping a single-tenant billing system. Server-sent events for live dashboard updates are an optional P2, kept only if time allows.

Repository layout

```
brightloop/
├─ apps/
│  ├─ api/             Express API, Prisma schema, scheduler
│  └─ web/             React app (staff, admin and customer portal)
├─ packages/shared/    Zod schemas, types, pricing and tax engine
├─ docs/               PRD, ERD, OpenAPI export
├─ docker-compose.yml  PostgreSQL and Mailpit
└─ .github/            CI workflow, PR template, CODEOWNERS
```

The pricing, discount and tax engine is a pure function in `packages/shared`, so the subscription screens, the billing run and the tests all use the same code.

10. Team split (4 members)

Work is divided into four vertical slices: each member builds the database tables, API and screens for their modules, so nobody waits on a separate "backend person" or "frontend person". Replace the member labels with real names.

Member	Slice	Owns (API and UI)	Requirement IDs
Member 1: Platform and Access	Foundation	Repo, Docker Compose, CI, Prisma setup; signup, login, password reset, roles and route guards; staff management; customers; app shell, design tokens and shared UI components; customer portal shell	AUTH-1 to 4, SYS-1 (admin and base seeds)
Member 2: Catalog and Pricing	Setup data	Products and variants; plans and their switches and actions; tax rules; discount rules; the pricing, discount and tax engine in `packages/shared` with unit tests	CAT-1, 2, PLAN-1 to 3, TAX-1, DISC-1, 2
Member 3: Subscriptions and Billing	Core engine	Quotation templates; subscriptions with lines and lifecycle; billing run and scheduler; invoice generation, lifecycle and discount application; idempotency tests	QT-1, SUB-1 to 3, INV-1, 2, 4, DISC-3, PLAN auto-close
Member 4: Payments, Reports and Portal	Money and insight	Invoice send, print and PDF; payments and overdue logic; finance dashboard, filters, summary, MRR, CSV; customer portal pages and actions; bulk seed and load test; README and demo script	INV-3, PAY-1, 2, REP-1 to 3, PORT-1, 2, SYS-1 (bulk), SYS-2, 3

Everyone also owns

- Unit and integration tests for their slice, and the OpenAPI entries for their endpoints.
- Seed rows for their own tables.
- Reviewing at least two teammates' pull requests; review pairs are 1 ↔ 2 and 3 ↔ 4, with the other pair as backup.

- Presenting their own slice in the final demo.

Hand-offs (what blocks whom)

From	To	What	Needed by
Member 1	All	Running repo, database, auth guard, app shell and shared components	End of Phase 1
Member 2	Member 3	Pricing, discount and tax engine plus plans and products in the database	Middle of Phase 2
Member 3	Member 4	Invoice table filled by the billing run	Start of Phase 3
Member 4	Member 3	Bulk seed data for testing the billing run at scale	Phase 3

Until an upstream slice lands, work against the agreed schema and seed rows in the local database rather than static JSON in the UI.

Day-0 session (all four, 15 minutes): agree the Prisma schema, the OpenAPI skeleton, the status names and the design tokens. Nothing else starts before this is committed.

11. Git workflow and collaboration

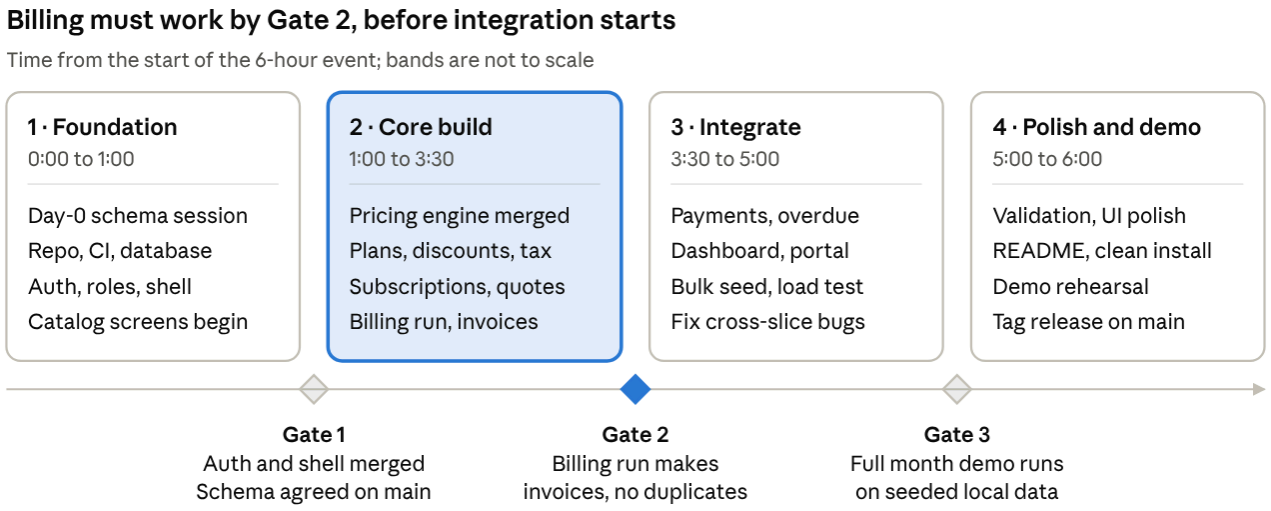
All four members are collaborators on one shared repository and every member commits, opens pull requests and reviews, so the history shows four contributors.

- **Branches**: `main` is protected and always runs; work happens on `feature/<slice>-<topic>` (for example `feature/billing-run`) or `fix/<topic>`. No direct pushes to `main`.
- **Pull requests**: small (under about 400 changed lines), linked to a requirement ID, approved by one teammate who did not write it, with CI passing (lint, type check, tests) before merge.
- **Commits**: Conventional Commits such as `feat(billing): create invoice per due subscription`; commit often, at least every 30 minutes per member.
- **Merge rhythm**: merge to `main` every 45 to 60 minutes and rebase feature branches from `main` every hour to avoid late conflicts.
- **Ownership**: a CODEOWNERS file maps folders to slice owners; shared files (`schema.prisma`, `packages/shared`) need a second reviewer.
- **Schema changes**: one migration per pull request, announced in the team chat before merge; never edit a migration that is already on `main`.

- **Tracking**: one GitHub issue per requirement ID on a Projects board (To do, In progress, In review, Done), assigned to its slice owner.
- **Hygiene**: `.env.example` is committed, real `.env` is ignored; the README has a clone-to-running guide that every member tests on a clean machine.

12. Timeline, risks and hackathon checklist

The build runs in four phases with three gates, sized for a 6-hour hackathon, so every phase is short and scope is trimmed to the P0 path.



roadmap · 4 phases, 3 gates

Phase 2 is highlighted because everything in Phases 3 and 4 depends on the billing run producing correct invoices.

Risks

Risk	Mitigation
Billing run is wrong or creates duplicate invoices	Unique constraint, one transaction per subscription, integration test that runs billing twice; Member 2 reviews Member 3's billing code
Schema changes break teammates' work	Day-0 schema session, one migration per pull request, announced in chat
Member 3's slice is the heaviest	Member 2 hands over the pricing engine early and pairs on invoice lines; P1 items are cut first
Dashboard misses the 2-second target	Indexes, SQL aggregates, pagination; load test in Phase 3

Risk	Mitigation
Docker is unavailable on a laptop	Documented fallback to a local PostgreSQL install; console logging if Mailpit is missing
Merge conflicts in shared files	CODEOWNERS, second reviewer, merge to `main` every hour
Time runs out	Cut P2, then P1; the P0 list in section 3 is the demo

Out of scope for v1

- Online payment gateway (payments are recorded by staff), refunds and credit notes.
- Multi-currency, multi-tenant setups, tax by jurisdiction beyond admin-defined rules.
- Proration for mid-period changes and stacking several discounts on one line.
- Native mobile apps (the web app is responsive instead).

Assumptions to confirm

- The hackathon lasts 6 hours and the team is free to choose its stack.
- The plan's price is a recurring base fee billed as its own invoice line, with product lines added on top.
- One currency, with the symbol set in one configuration value.
- A customer record created by staff is linked to a portal login when someone signs up with the same email.

Hackathon rules checklist

Rule	How the project meets it	Owner
Real or dynamic data, not static JSON	Every screen reads from the REST API and PostgreSQL; static JSON only in the first 30 minutes; dashboard refreshes by polling	All; Member 4 for the dashboard
Responsive, clean, consistent UI	Design tokens, shared component kit, three breakpoints, one app shell	Member 1
Robust input validation	Shared Zod schemas in browser and server, rules in section 8, field-level API errors	All; Member 1 owns the shared validation set-up

Rule	How the project meets it	Owner
Intuitive navigation and spacing	Role-based sidebar and menu groups (section 7), 8 px spacing scale	Member 1
Proper Git use by everyone	Protected `main`, cross-reviewed pull requests, slice branches, daily commits, issue board	All
Backend APIs, data model, local database	Sections 5 and 6, Prisma migrations, seed script	Member 1 for set-up; each member for their tables and endpoints
Understand code before using it	Every author can explain their pull request; snippets and AI output are adapted to project names and rules and checked in review	All
Offline or local solutions	Docker Compose or local PostgreSQL, Mailpit, bundled fonts and libraries, local PDF and print	Member 1
Trendy technology only with real value	No microservices, GraphQL or AI features; server-sent events stay optional	All